

PELAN PEMBANGUNAN PERISIAN

(SOFTWARE BLUEPRINT)

Projek ArchiVox: Web-Based Voxel Architecture Game

Dokumen:	Spesifikasi Teknikal & Panduan Arkitektur (Main Reference)
Fasa:	Pra-Pembangunan (Pre-development Planning)
Teknologi Utama:	Laravel 11, React, Three.js / React Three Fiber, Docker, MySQL
Status:	Sedia untuk Pelaksanaan (Ready for Implementation)

1. Pengenalan Projek & Visi Produk

Projek ArchiVox merupakan sebuah permainan video berasaskan laman web (web-based browser game) yang direka khas untuk komuniti arkitek dan pereka bentuk ruang. Berasaskan **Concept A (The Grid & Form)**, fokus utama permainan ini adalah memberikan kepuasan estetika, gubahan rupa bentuk (form-making), komposisi spatial, dan interaksi cahaya-bayang tanpa tekanan objektif kalah/menang yang ketat.

Sistem ini menggunakan algoritma generasi prosedur (procedural generation) peringkat ringan untuk menterjemahkan interaksi klik pengguna kepada struktur seni bina yang kontekstual secara real-time. Melalui integrasi penuh persekitaran Docker, ekosistem pembangunan lokal akan diselaraskan untuk memastikan skalabiliti kod antara frontend grafik (React + Three.js) dan backend pengurusan data (Laravel API).

2. Seni Bina Sistem (System Architecture)

Aplikasi ini menggunakan pendekatan decoupled architecture (分離型 architecture) di mana bahagian frontend dan backend dipisahkan sepenuhnya untuk mengoptimumkan prestasi render browser dan kecekapan pemprosesan data.

2.1 Struktur Komponen

- **Lapisan Persembahan (Frontend):** Memanfaatkan React sebagai pengurus state UI dan React Three Fiber (R3F) sebagai lapisan abstraksi WebGL/Three.js untuk pemprosesan grafik 3D secara perkakasan (hardware-accelerated).

- **Lapisan Aplikasi (Backend):** Memanfaatkan Laravel sebagai Stateless RESTful API untuk mengendalikan pengesahan pengguna (authentication), penyimpanan sesi log masuk, pengurusan galeri projek, dan fungsi CRUD skema voxel.
- **Lapisan Data (Database):** MySQL digunakan untuk menyimpan metadata pengguna dan struktur spatial voxel dalam format notasi objek JavaScript (JSON) termampat.

3. Spesifikasi Persekitaran Docker (Local Development Setup)

Bagi memastikan keselarasan persekitaran pembangunan antara ahli pasukan, keseluruhan stack perisian akan dijalankan di dalam kontena Docker. Infrastruktur ini dipecahkan kepada empat perkhidmatan utama melalui fail `docker-compose.yml`.

3.1 Fail Konfigurasi Utama: docker-compose.yml

```

version: '3.8'

services:
  # Perkhidmatan Backend & API (Laravel)
  laravel-app:
    build:
      context: ./backend
      dockerfile: Dockerfile
    image: archivox-backend
    container_name: archivox_backend_app
    restart: unless-stopped
    tty: true
    environment:
      SERVICE_NAME: app
      SERVICE_TAGS: dev
    working_dir: /var/www
    volumes:
      - ./backend:/var/www
      - ./backend/php/local.ini:/usr/local/etc/php/conf.d/local.ini
    networks:
      - archivox-network

  # Perkhidmatan Web Server (Nginx)
  nginx:
    image: nginx:alpine
    container_name: archivox_nginx
    restart: unless-stopped
    ports:
      - "8000:80"
    volumes:
      - ./backend:/var/www
      - ./nginx/conf.d:/etc/nginx/conf.d
    networks:
      - archivox-network

  # Perkhidmatan Frontend (React Vite)
  react-frontend:
    image: node:20-alpine
    container_name: archivox_frontend
    working_dir: /app
    volumes:
      - ./frontend:/app
    ports:
      - "3000:3000"
    command: sh -c "npm install && npm run dev -- --host"
    networks:
      - archivox-network

  # Perkhidmatan Pangkalan Data (MySQL)

```

```
mysql-db:
  image: mysql:8.0
  container_name: archivox_mysql
  restart: unless-stopped
  ports:
    - "3306:3306"
  environment:
    MYSQL_DATABASE: archivox_db
    MYSQL_ROOT_PASSWORD: root_secret_password
    MYSQL_USER: archivox_user
    MYSQL_PASSWORD: user_secret_password
  volumes:
    - dbdata:/var/lib/mysql
  networks:
    - archivox-network

networks:
  archivox-network:
    driver: bridge

volumes:
  dbdata:
    driver: local
```

3.2 Dockerfile untuk Laravel Backend

```
FROM php:8.3-fpm-alpine

# Pasang dependency sistem luar
RUN apk update && apk add --no-cache      build-base      shadow      curl      git
bash      libpng-dev      libjpeg-turbo-dev      freetype-dev      libzip-dev      zip
unzip

# Pasang extension PHP yang diperlukan
RUN docker-php-ext-configure gd --with-freetype --with-jpeg      && docker-php-ext-
install pdo_mysql gd zip bcmath

# Dapatkan Composer versi rasmi terbaru
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# Tetapkan direktori kerja
WORKDIR /var/www

# Salin kebenaran folder asal
RUN chown -R www-data:www-data /var/www

USER www-data

EXPOSE 9000
CMD ["php-fpm"]
```

4. Reka Bentuk Pangkalan Data & Struktur Data Voxel

Cabaran utama Concept A ialah cara menyimpan susunan grid 3D tanpa membebankan pangkalan data. Kita akan mengabaikan penyimpanan fail model 3D biner (seperti .obj) dan sebaliknya menyimpan struktur data matriks koordinat diskret dalam bentuk JSON.

4.1 Skema Migrasi Laravel: Tabela `projects`

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('projects', function (Blueprint $table) {
            $table->id();
            $table->foreignId('user_id')->constrained()->onDelete('cascade');
            $table->string('title', 100);
            $table->string('slug')->unique();
            $table->json('voxel_data'); // Menyimpan array koordinat spasial [{x, y, z,
type}]
            $table->string('color_palette', 30)->default('brutalist_raw');
            $table->decimal('time_of_day', 4, 2)->default(17.00); // Kedudukan matahari
(0.00 - 23.59)
            $table->boolean('is_public')->default(true);
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('projects');
    }
};
```

4.2 Contoh Struktur Payloads Data JSON (Voxel Data Schema)

Setiap kali fungsi `Save` dipanggil dari frontend React, struktur array objek berikut akan dihantar melalui API HTTP POST:

```
[
  {"x": 0, "y": 0, "z": 0, "type": "foundation"},
  {"x": 0, "y": 1, "z": 0, "type": "wall_window"},
  {"x": 0, "y": 2, "z": 0, "type": "roof_terrace"},
  {"x": 1, "y": 0, "z": 0, "type": "foundation"},
  {"x": 1, "y": 1, "z": 0, "type": "balcony"}
]
```

5. Algoritma Generasi Prosedur (Neighbour Checking Engine)

Rupa bentuk geometri bagi setiap blok ditentukan secara dinamik melalui penilaian keadaan nod jiran (neighbourhood state analysis). Sistem koordinat menggunakan pemetaan integer tiga dimensi (x, y, z) .

Aturan Asas Penentuan Struktur Voxel:

1. **Syarat Asas (Foundation):** Jika nilai $y = 0$, jenis voksel diwajibkan menjadi tapak konkrit atau struktur asas (foundation).
2. **Syarat Bumbung (Roof):** Untuk koordinat (x, y, z) , jika tiada entiti wujud pada posisi atas menegak $(x, y+1, z)$, maka jenis komponen akan ditukarkan kepada bumbung (roof profile).
3. **Syarat Dinding/Tingkap (Wall/Window):** Jika terdapat elemen pada $(x, y+1, z)$, rupa asal bumbung secara automatik berubah menjadi dinding tengah. Jika nod sisi $(x+1, y, z)$ atau $(x-1, y, z)$ adalah kosong, bukaan tingkap (aperture) akan dijanakan pada permukaan luar tersebut.

5.1 Fungsi Teras Penilaian State (Frontend Implementation)

```
function evaluateVoxelMesh(currentVoxel, allVoxels) {
  const { x, y, z } = currentVoxel;

  // Imbas keberadaan jiran atas
  const hasTopNeighbour = allVoxels.some(v => v.x === x && v.y === y + 1 && v.z === z);
  // Imbas keberadaan jiran bawah
  const hasBottomNeighbour = allVoxels.some(v => v.x === x && v.y === y - 1 && v.z === z);

  if (y === 0) {
    return 'FOUNDATION_MESH';
  }

  if (!hasTopNeighbour) {
    return 'ROOF_MESH';
  }

  if (hasTopNeighbour && hasBottomNeighbour) {
    return 'WALL_INTERMEDIATE_MESH';
  }

  return 'GENERIC_BLOCK';
}
```


6. Pelan Tindakan & Fasa Pembangunan (Roadmap)

Pelaksanaan projek ini dibahagikan kepada empat fasa utama sepanjang tempoh 8 minggu pembangunan lokal:

Fasa	Minggu	Fokus Utama & Deskripsi Tugas	Milestone Penyerahan
Fasa 1	Minggu 1-2	Konfigurasi persekitaran Docker. Penyediaan boilerplate Laravel API dan infrastruktur awal pangkalan data MySQL. Setup bundler Vite React.	Docker Container Berjalan Lancar & Koneksi API Berjaya.
Fasa 2	Minggu 3-4	Pembangunan sistem kanvas WebGL menggunakan React Three Fiber. Implementasi fungsi orbit camera, snapping raycaster untuk grid lantai 3D.	Pengguna Boleh Klik Dan Meletakkan Kotak 3D Di Atas Grid.
Fasa 3	Minggu 5-6	Integrasi algoritma "Neighbour Checking". Pembangunan logik pertukaran bumbung, dinding dan tangga secara dinamik. Tambahan slider waktu dan bayang.	Sistem Grafik Bertukar Rupa Secara Automatik (Procedural Core Live).
Fasa 4	Minggu 7-8	Penyambungan fungsi API Save/Load ke Laravel Backend. Pengoptimuman prestasi render buffer 3D. Fasa ujian bug (UAT) bersama pengguna sasaran.	Pelancaran Sistem Versi Web Beta Sedia Untuk Digunakan.

Saranan Pengoptimuman Masa Hadapan:

Bila bilangan blok melebihi 1,000 unit, manipulasi state DOM React biasa akan mula mengalami penurunan kadar bingkai (frame drop). Disarankan untuk mengimplementasikan teknik **Instanced Meshes** dalam Three.js pada Fasa 4 untuk memastikan satu panggilan draw (single draw call) bagi geometri yang serupa bagi mengekalkan prestasi 60 FPS pada peranti mudah alih.